

COGNITIVEOS

DOCUMENTATION FIRST
PRE-BUILD SPEC

A **cognitive load simulator**, not another to-do app. Lay out a day of tasks, meetings, alerts and deadlines; CognitiveOS estimates minute by minute how loaded your mind is, shows **what is driving it**, and lets you test a different way of working before you live it.

GROUNDED IN

COGNITIVE LOAD & INTERRUPTION RESEARCH

19 peer-reviewed sources, appendix B

FIELDS

PSYCHOLOGY · HCI · AI

Behavioural analytics, visualisation

STATUS

DESIGN DRAFT

Build starts after sign-off

CONTENTS

WHAT'S INSIDE

01	PROJECT OVERVIEW	the problem, the idea, who it's for
02	THE SCIENCE	six drivers of load, and the evidence for each
03	THE MODEL	equations, zones, calibration
04	WHAT-IF EXPERIMENTS	the prototype run on a sample day
05	WHAT WE'LL BUILD	every module, MVP and later
06	SCENARIO LIBRARY	six people, six levers
07	SCREENS WE'LL DESIGN	wireframes of the four main views
08	TECH STACK	frontend, engine, storage, AI
09	ARCHITECTURE	layers, tick loop, data model
10	PRINCIPLES	honesty, privacy, determinism
11	BUILD PLAN	six weeks, milestone by milestone
12	FUTURE FEATURES	what could come after v1
A·B	APPENDICES	glossary, design language, references

6 LOAD DRIVERS

19 CITED STUDIES

1 min SIMULATION STEP

0 TRACKERS OR ACCOUNTS

HOW TO READ THIS DOCUMENT

This is written **before any code**, so it is a specification and a plan, not a report. Citations in red (for example Leroy 2009) point to appendix B. Every coefficient the model uses is either tied to a source or tagged STAND-IN, meaning we picked a sensible value that calibration will replace. The charts in sections 03 and 04 come from a 90-line prototype of the model, run on a made-up sample day, so they show **how the simulator will behave**, not findings about real people. Items tagged MVP are committed for v1; V2 and LATER are planned, not promised.



MODEL THE MIND, NOT THE TO-DO LIST

Productivity apps count tasks. But two days with the same task list can feel completely different: one with long quiet blocks, one chopped up by meetings and pings. CognitiveOS models that difference. It treats a day as a sequence of demands on limited working memory and simulates how they add up.

THE PROBLEM

- Knowledge work is fragmented. In one field study, people spent on average about **11 minutes** on a project before switching or being interrupted, and **57%** of those stretches were interrupted Mark et al. 2005.
- The costs are invisible. A switch leaves **attention residue** on the old task Leroy 2009; an alert hurts focus even if you never pick up the phone Stothart et al. 2015.
- Tools show *what* you did, not *why* the day felt heavy, and they never let you try a different pattern before committing to it.

OUR ANSWER

- A **transparent load model**: six drivers, each tied to published research, combined with weights you can see and change.
- A **simulator, not a tracker**. You describe a day; nothing watches you. Then you ask "what if I batch my notifications?" and watch the curve change.
- **Exact explanations**. Because the model is additive, every minute splits cleanly into its drivers. No black box to reverse-engineer.
- **Personal calibration** from short NASA-TLX check-ins, with the uncertainty shown.

WHO IT'S FOR

Students planning exam weeks and group projects. **Developers and designers** whose deep work keeps getting cut by meetings. **Team leads** who want to understand the cost of the interruptions they create, on their own schedule, not anyone else's. **HCI and psychology learners** who want to see interruption research run as a model rather than read it as a list of findings.

A USER'S JOURNEY

1 · BUILD A DAY
Drag tasks and meetings onto a timeline, rate how complex each is.

2 · ADD THE NOISE
Alert rate, open tasks, deadlines, planned breaks.

3 · SIMULATE
Minute-by-minute load curve, coloured by driver.

4 · ASK WHY
Click a spike: which drivers, which events caused it.

5 · TRY A WHAT-IF
Pull a lever, compare the two days side by side.

6 · CALIBRATE
Rate real days with NASA-TLX; the weights become yours.

WHAT IT IS NOT

Not a diagnostic tool, not a measure of anyone's ability, and not a way for employers to monitor staff. It produces an **estimate under stated assumptions**, and says so on every screen.



SIX THINGS THAT LOAD A MIND

Working memory holds only about four chunks at once Cowan 2001. Everything below competes for that space. For each driver: what happens, the evidence, and how CognitiveOS turns it into a number.

x_1 · intrinsic

TASK COMPLEXITY

Tasks whose parts must be held in mind together impose more load than tasks that can be done piece by piece Sweller 1988. Cognitive Load Theory calls this **intrinsic load** and separates it from load added by how work is presented Sweller et al. 1998.

HOW WE MODEL IT

Each block gets a complexity rating c from 1 (sorting email) to 5 (debugging a race condition). $x_1 = c / 5$ while working on it.

x_2 · switching

SWITCHING & RESIDUE

Switching between tasks costs time even when the switch is predictable Rogers & Monsell 1995, and the cost grows with how complex the tasks are Rubinstein et al. 2001. Leaving a task unfinished keeps part of your attention on it: **attention residue** Leroy 2009. Resuming means re-activating the old goal Altmann & Trafton 2002.

HOW WE MODEL IT

A spike at each switch (decay $\tau = 3$ min) plus residue proportional to the old task's complexity ($\tau = 12$ min).

STAND-IN TS

x_3 · alerts

NOTIFICATIONS

Just receiving a phone notification, without answering it, impaired performance on a sustained-attention task about as much as using the phone did Stothart et al. 2015. In a field experiment, people whose notifications were batched three times a day reported more focus and less stress Fitz et al. 2019.

HOW WE MODEL IT

Each alert adds a short spike ($\tau = 1.5$ min). With probability $p = 0.35$ you act on it: a 3-minute detour that counts as a switch. Batching holds alerts until set times.

x_4 · open loops

OPEN LOOPS

Unfinished goals keep intruding on unrelated work. Making a specific plan for the goal, without finishing it, removed that interference Masicampo & Baumeister 2011.

HOW WE MODEL IT

Background load that saturates with the number of open tasks n: $1 - e^{-n/4}$. Halved when each task has a written next step.

x_5 · time

DEADLINE PRESSURE

Temporal demand is one of the six dimensions of the NASA-TLX workload scale Hart & Staveland 1988. Interrupted people finished tasks faster but reported more stress, frustration, time pressure and effort Mark et al. 2008.

HOW WE MODEL IT

Work left ÷ time left for each deadline, ramping in over the last 4 hours. The tightest deadline wins.

x_6 · fatigue

FATIGUE

Mental fatigue builds with time on a demanding task and reduces engagement Boksem & Tops 2008. Brief, occasional breaks from a task prevented the usual drop in vigilance Ariga & Lleras 2011.

HOW WE MODEL IT

An accumulator that grows faster under high load and recovers during breaks ($\tau = 14$ min). It lags, so a heavy morning shows up after lunch.

ONE NUMBER, SIX HONEST PARTS

Load at minute t is a weighted sum of the six drivers. Simple on purpose: a sum can be explained exactly, tested for sensible behaviour, and fitted to a person with very little data.

```
// load at minute t, from the six drivers in section 02
L(t) = w₁·complexity + w₂·switching + w₃·alerts + w₄·openLoops + w₅·deadline + w₆·fatigue
switching(t) = Σ_switches s e^-(t-s)/3 + 0.75 · Σ_s (c_prev/5) · e^-(t-s)/12
fatigue(t+1) = F + (0.4 + L)(1 − F)/260 while working · F · e^-1/14 during breaks
// default weights (stand-ins until calibration): w = (0.34, 0.30, 0.16, 0.14, 0.14, 0.22)
```

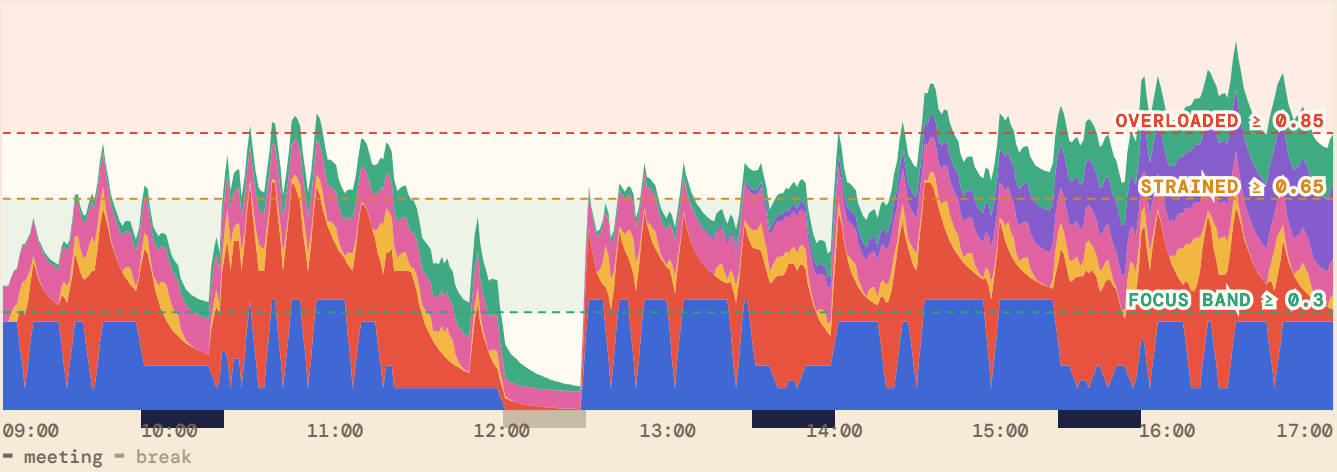
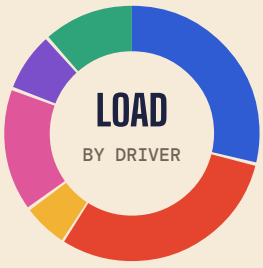


FIG. 1 · Prototype run on the sample day (09:00-17:00, three meetings, four tasks, 113 alerts, six open tasks, a 17:00 deadline). Each colour is one driver's share of the load.



Task complexity	29%
Switching & residue	30%
Notifications	6%
Open loops	16%
Deadline pressure	8%
Fatigue	12%

READING THE SAMPLE DAY

Mean load is **0.67** and **117 minutes** sit above 0.85. Switching and residue account for **30%** of all load, as much as the work itself: the late-morning spikes are switches between code and the report, not hard tasks.

ZONES, STATED CAREFULLY

We shade four bands: under-loaded (< 0.3), focus band, strained (≥ 0.65), overloaded (≥ 0.85). They echo the inverted-U idea that performance peaks at moderate arousal Yerkes & Dodson 1908, but that "law" has been restated so many ways that it can't be treated as one Teigen 1994. So the bands are a display convention that calibration moves, not a claim.

CALIBRATION

After a real block of work, a 30-second check-in asks the six raw NASA-TLX questions Hart 2006. From 8 check-ins on, we fit personal weights by non-negative ridge regression, pulled towards the defaults, and show a bootstrap interval around every curve. Fewer check-ins: defaults, labelled as such.

TRY THE DAY BEFORE YOU LIVE IT

The point of a simulator is the counterfactual. Same tasks, same total work, a different arrangement. We ran the prototype on the sample day with three levers from the research, alone and together. These numbers come from stand-in parameters on an invented day: they show the simulator moves in the direction the studies predict, not real effect sizes.

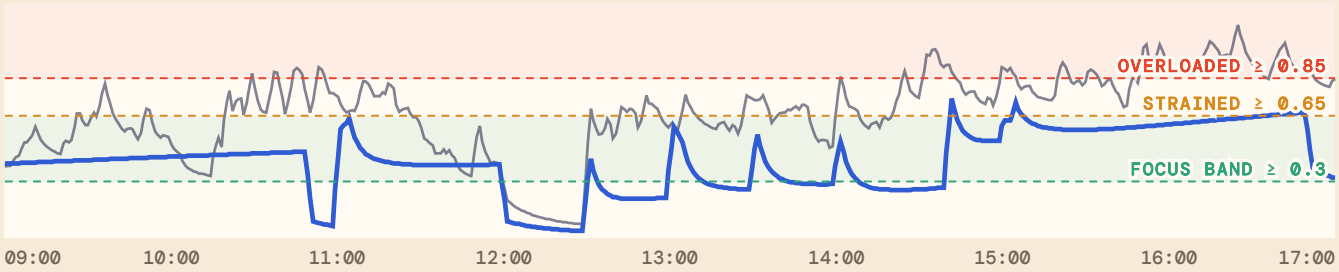


FIG. 2 · Load on the sample day as planned (grey) and with all three levers applied (blue).

VARIANT	MEAN LOAD	PEAK	FOCUS MIN	STRAINED MIN	OVERLOADED MIN	UNDER MIN
As planned The sample day: three meetings, four tasks, alerts as they arrive	0.67	1.19	164	169	117	30
Batch alerts Notifications held and delivered 4× a day (Fitz et al. 2019)	0.53 -20%	1.02	281	145	12	42
Write plans A written next step for every open task (Masicampo & Baumeister 2011)	0.62 -8%	1.13	207	164	66	43
Cluster meetings All meetings after lunch, deep work in long morning blocks	0.61 -9%	1.18	242	128	79	31
All three Clustered meetings + batched alerts + written plans	0.41 -39%	0.82	322	25	0	133

BATCHING WINS ALONE

In this model an acted-on alert becomes a switch, so batching removes dozens of small switches. That's why it beats the other two levers on its own. Whether that holds for a real person is what calibration finds out.

LEVERS STACK

Together the three levers do more than any one: long morning blocks leave fewer switches for alerts to multiply. The What-If Lab shows each lever's share of the gain, so the user knows which habit is worth keeping.

TOO CALM IS A RESULT TOO

The combined day moves a lot of time into the under-loaded band. CognitiveOS reports that instead of treating lower as always better. An under-loaded afternoon is a cue to schedule the hard task there.



EVERY MODULE, IN ORDER OF NEED

Twelve modules. The MVP is a complete loop on its own: build a day, simulate it, understand it, change it, calibrate it.

MODULE	WHAT IT DOES	SCOPE
Day Builder	Drag work blocks, meetings and breaks onto a timeline. Rate each task's complexity 1–5, set alert rate, open tasks and deadlines.	MVP
Simulation Engine	Minute-step model from section 03, deterministic from a seed, running in a Web Worker so the UI never stalls.	MVP
Load Dashboard	Stacked driver chart with zone bands, peak markers, minutes per zone and the driver donut.	MVP
"Why?" Explainer	Click any minute: exact driver shares plus the events behind them ("switch from Code at 11:05 is still leaving residue").	MVP
What-If Lab	Levers: batch alerts, write plans, cluster meetings, add breaks, drop a task, lower alert rate. Overlay and diff table.	MVP
Scenario Library	Six ready-made days (section 06) to explore without entering anything personal.	MVP
Check-in	Six raw NASA-TLX sliders after a real block of work, stored locally against that block.	MVP
Model Card	One page listing every parameter: value, source, stand-in flag, and what changing it does.	MVP
Calibration	Fits personal weights from check-ins with intervals; shows how far they moved from defaults and how sure we are.	V2
Schedule Search	Searches reorderings of movable blocks (meetings stay fixed) for the fewest overloaded minutes, and explains the winner.	V2
AI Narrator	Opt-in. A language model turns the attribution data into a short plain-language summary. Every number it writes is checked against the data; if one doesn't match, the summary is dropped.	V2
Calendar import & export	Import a day from an .ics file; export a day report as PNG or PDF.	LATER

WHERE THE "AI" IS, HONESTLY

The core is an interpretable model, not a neural network, because a load estimate you can't explain is worse than none. The AI parts sit around it: **learning personal weights** from check-ins, **searching** for better schedules, and **narrating** results. None of them can change what the model says happened.



SIX PEOPLE, SIX LEVERS

Each scenario is a ready-made day plus one lever to try. The "expect" line is what the model should show if it's built right; these double as end-to-end tests.

S1 · STUDENT

EXAM WEEK



Four subjects in one day, phone face-up, about 30 alert hour from group chats, a lab report due at 23:59.

LEVER Batch alerts to the top of each hour. EXPECT switching share falls; deadline pressure unchanged.

S2 · DEVELOPER

THE CHOPPED-UP MAKER



Four 30-minute meetings spread evenly; no gap longer 70 minutes for complexity-5 work.

LEVER Cluster meetings after lunch. EXPECT fewer strained minutes in the morning; a dip in the meeting block.

S3 · ENGINEER ON CALL

PAGER DAY



A high alert rate where most alerts must be acted on (p 0.8), on top of a planned feature task.

LEVER Hand the pager over for a 2-hour block. EXPECT one clean focus window; the model shows what the hand-over buys.

S4 · TEAM MEMBER

THE JUGGLER



Nine open tasks across three group projects, none finish today, each touched briefly.

LEVER Write a next step for every open task. EXPECT open-loop load halves; switching unchanged.

S5 · RESEARCHER

THE MARATHON



One complexity-4 task for five hours straight, few alerts breaks.

LEVER A 5-minute break every 50 minutes. EXPECT fatigue stays lower all afternoon at the cost of 25 minutes of work.

S6 · FREELANCER

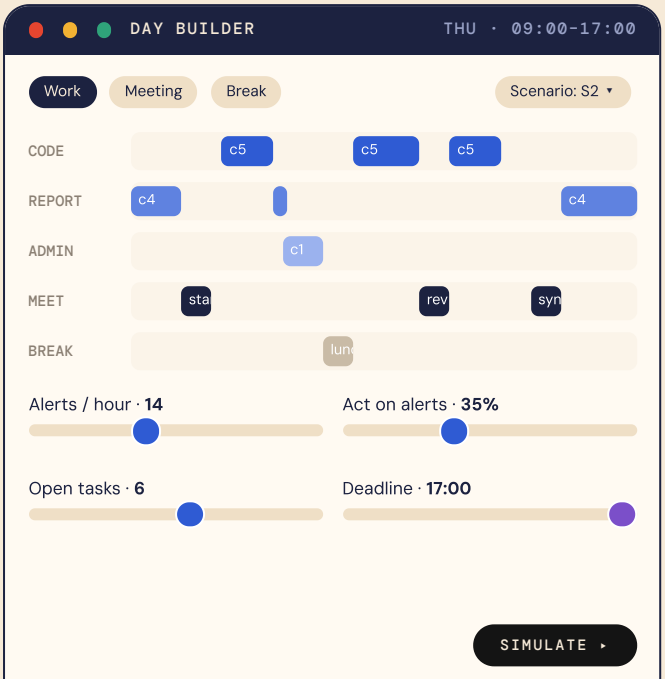
DEADLINE CRUNCH



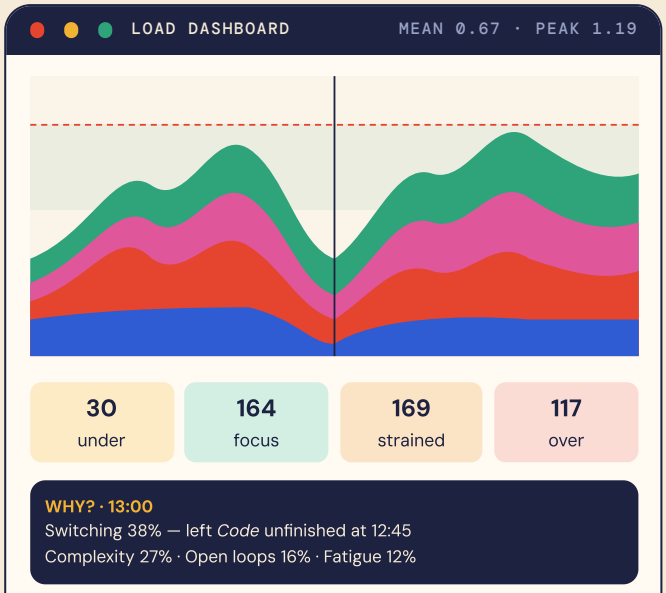
Three client projects, two due tomorrow, more work left hours in the day.

LEVER Drop or renegotiate one project. EXPECT deadline pressure and open loops both fall; shows the cost of over-committing.

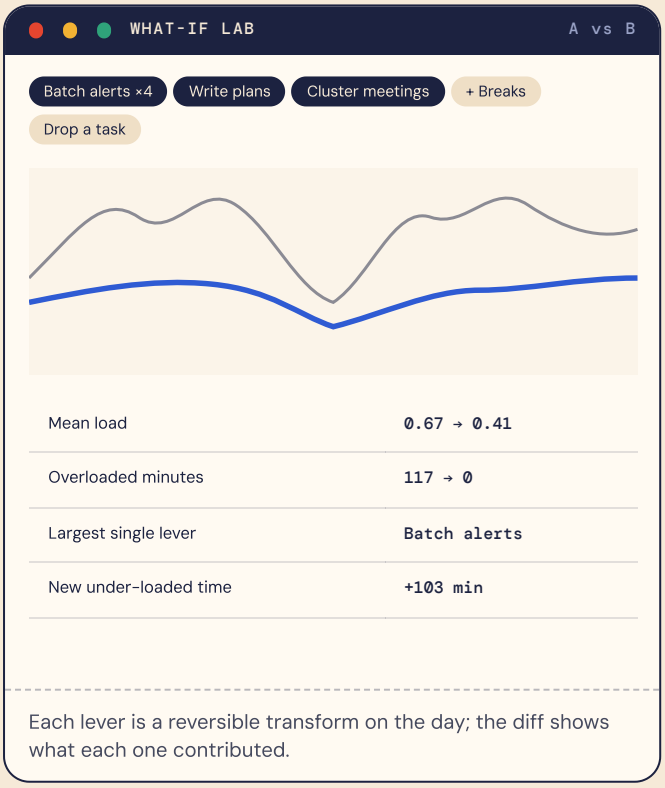
FOUR VIEWS



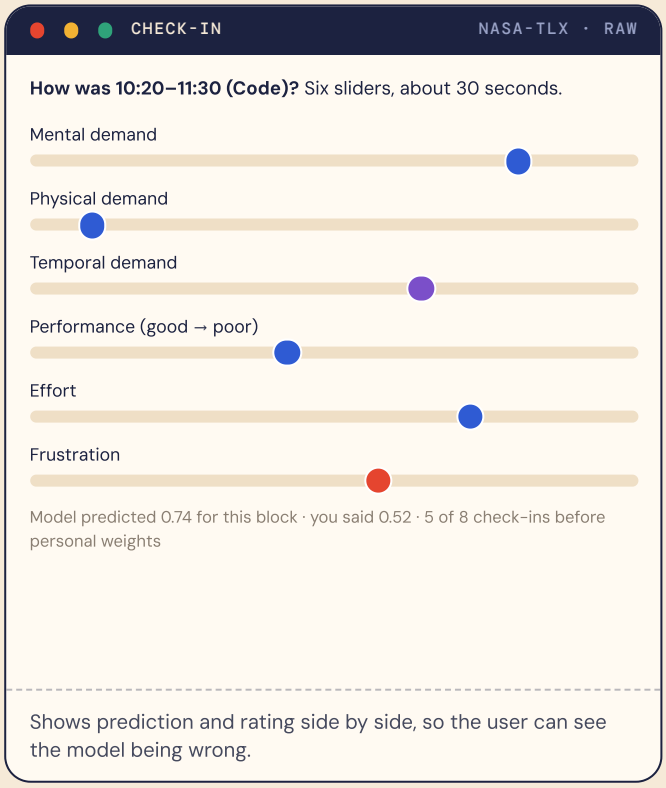
Blocks snap to 5 minutes. Complexity is set by a 1–5 chip on each block; colour depth follows it.



The cursor line pins a minute; the "Why?" card updates live. Zone counts are in minutes.



Each lever is a reversible transform on the day; the diff shows what each one contributed.



Shows prediction and rating side by side, so the user can see the model being wrong.



BROWSER-FIRST, LOCAL-FIRST

Everything runs on the user's device. A day is a small JSON document and a simulation is a few hundred thousand arithmetic operations, so there's no reason to send either to a server. The only network call is the optional AI narrator.

LAYER	CHOICE	WHY
UI	React 19 · TypeScript · Vite	Same toolchain as SignalSafe, so components, lint and CI setup carry over.
State	Zustand	Small store for the day being edited, the active what-if and the selected minute.
Timeline editing	dnd-kit	Accessible drag and drop with keyboard support, needed for snapping blocks.
Charts	D3 scales & shapes → React SVG	Stacked areas, donuts and overlays with full control over the design language; SVG exports cleanly.
Engine	Pure TypeScript in a Web Worker (Comlink)	No DOM, no dates, no randomness except a seeded PRNG: easy to test and identical on every machine.
Schemas	Zod	One schema validates days, scenarios and imported files, and generates the TypeScript types.
Storage	IndexedDB via Dexie	Days, check-ins and fitted weights stay in the browser. Export and import as JSON.
Fitting	Small NNLS ridge solver in TS	Six weights, a few dozen check-ins: no ML library needed, and the fit is fully inspectable.
AI narrator	Claude API (Haiku 4.5) via a serverless proxy	Opt-in. Only the attribution summary is sent, never the raw day. The proxy keeps the API key off the client.
Testing	Vitest · fast-check · Playwright	Unit and property tests for the engine; scenario tests from section 06; end-to-end tests for the main flows.
Delivery	GitHub Actions · Vercel	Tests and type checks on every push; preview deploy per pull request.

WHAT WE DELIBERATELY SKIP

No accounts, no backend database, no analytics SDK, no browser extension that watches activity. Each would make the tool more powerful and less trustworthy.

PERFORMANCE BUDGET

A full 16-hour day (960 ticks) simulates in under 20 ms, so dragging a block re-runs the model live. Schedule Search runs thousands of these in the worker.



FOUR LAYERS, ONE DIRECTION

Data flows down as a day description and back up as a trace. The engine never knows there is a UI, which is what lets the same code power the dashboard, the What-If Lab, the tests and Schedule Search.

PRESENTATION · REACT

Day Builder · Load Dashboard · "Why?" card · What-If Lab · Check-in · Model Card · Scenario picker

▼ day JSON · lever list ▲ trace · attribution · summary

APPLICATION · ZUSTAND STORE + SERVICES

dayStore · levers → day transforms · attribution(minute) · calibration fit · narrator client (opt-in)

▼ postMessage via Comlink ▲ typed arrays

ENGINE · WEB WORKER, PURE TYPESCRIPT

params.ts (every value + source) · alert generator (seeded) · six driver functions · fatigue integrator · tick loop · schedule search

▼ save · load

STORAGE · INDEXEDDB (DEXIE)

days · check-ins · fitted weights + intervals · JSON export / import

DATA MODEL

```
type Block    = { from: number; to: number; task: string; kind: 'work' | 'meeting' | 'break'; c: 1 | 2 | 3 | 4
| 5 }
type Day      = { minutes: number; blocks: Block[]; alertsPerHour: number; actOnAlerts: number;
                batchAt: number[] | null; openTasks: { name: string; planned: boolean }[];
                deadlines: { at: number; workLeft: number }[]; seed: number }
type Tick     = { t: number; load: number; parts: Record<Driver, number>; events: Event[] }
type Lever    = (day: Day) => Day            // batch alerts, write plans, cluster meetings, add breaks…
type CheckIn  = { blockId: string; tlx: [number, number, number, number, number, number]; at: string }
type Param    = { value: number; source: string; standIn: boolean; note: string }
```

THE TICK LOOP

For each minute: find the active block, register switches and delivered alerts as events, evaluate the six drivers, weight and sum them, then advance fatigue. Events are kept so the "Why?" card can name the cause, not just the share.

LEVERS ARE PURE FUNCTIONS

A what-if never edits the user's day. It applies a list of `Lever` transforms to a copy, so variants can be combined, reordered and attributed one lever at a time.



BUILT TO BE TRUSTED

The weakest version of this project is a to-do app with a made-up score. These rules are what keep it from becoming that.

1 · EVERY NUMBER HAS A SOURCE

Each parameter lives in `params.ts` with its value, citation, a stand-in flag and a note. The Model Card renders that file directly, so the docs can't drift from the code.

2 · EXPLANATIONS ARE EXACT

Load is a sum, so driver shares are exact arithmetic, not an approximation like SHAP. The "Why?" card can't contradict the model because it is the model.

3 · UNCERTAINTY IS SHOWN

Default weights are labelled as defaults. Calibrated curves carry a bootstrap band. The check-in screen shows where the model was wrong.

4 · A SIMULATOR, NOT A DIAGNOSIS

No clinical words (burnout, ADHD, stress disorder), no scores about a person's ability, and nothing to hand to an employer. It's a planning aid for your own day.

5 · PRIVATE BY CONSTRUCTION

No accounts, no tracking, no server storage. The AI narrator is off by default; when on, it sees only the six-number summary, never task names.

6 · LOWER ISN'T ALWAYS BETTER

Under-load is reported as clearly as overload. The goal is time in the focus band, not an empty calendar.

PROPERTIES THE ENGINE MUST ALWAYS SATISFY

PROPERTY	TEST (FAST-CHECK, RANDOM VALID DAYS)
Deterministic	Same day + same seed → identical trace, bit for bit.
More alerts never help	Adding an alert never lowers load at any minute.
Breaks never add fatigue	Turning any work minute into a break never raises fatigue later.
Plans never hurt	Marking an open task as planned never raises open-loop load.
Shares add up	At every minute the six parts sum to the total within 1e-9.
Bounded	Every driver stays in [0, 1.2]; load is finite for days up to 24 h.



SIX WEEKS, ONE LOOP AT A TIME

Each week ends with something that runs. Week 2 already has a working headless simulator; everything after it is ways to see, change and trust it.

WEEK 1 · MODEL & RESEARCH

PARAMETERS WITH SOURCES

Read the 19 sources properly; write `params.ts` with every value, citation and stand-in flag. Port the prototype from this doc to TypeScript. Write the six scenario days as JSON.

DONE WHEN the Model Card can be generated from the file, and every scenario loads and validates.

WEEK 2 · ENGINE

DETERMINISTIC SIMULATOR IN A WORKER

Tick loop, seeded alert generator, six drivers, fatigue, event log. Property tests from section 10. Scenario "expect" lines as tests.

DONE WHEN all properties pass on 1,000 random days and a 16-hour day runs in under 20 ms.

WEEK 3 · DAY BUILDER

EDITING A DAY

Timeline with drag, resize and snap; complexity chips; alert, open-task and deadline controls; keyboard support; IndexedDB save and JSON import/export.

DONE WHEN each scenario can be rebuilt by hand, and edits re-simulate live.

WEEK 4 · DASHBOARD & WHY

SEEING AND UNDERSTANDING LOAD

Stacked driver chart, zones, donut, zone minutes; cursor and "Why?" card with events; Model Card page.

DONE WHEN clicking any spike names the event that caused it.

WEEK 5 · WHAT-IF LAB

COUNTERFACTUALS

Six levers as pure transforms; overlay chart; diff table with each lever's share of the change; scenario library screen.

DONE WHEN section 04's table can be reproduced in the app.

WEEK 6 · CHECK-IN, POLISH, SHIP

CLOSING THE LOOP

NASA-TLX check-in and prediction-vs-rating view; accessibility pass (contrast, keyboard, reduced motion); deploy; README and a portfolio case study written from this document.

DONE WHEN it's live, and a first-time visitor can go from a scenario to a what-if in under a minute.

After v1: Calibration fit, Schedule Search and the AI Narrator (all v2) need real check-in data to be worth building, so they come after a few weeks of using v1 on real days.



WHAT COULD COME NEXT

Planned in rough order of value. Anything that touches more personal data stays opt-in and local.

SCHEDULE SEARCH V2

Given fixed meetings and movable work, search arrangements for the fewest overloaded minutes and the most focus-band time on hard tasks. Simulated annealing over block orderings, all in the worker, with the winning schedule explained lever by lever.

PERSONAL CALIBRATION V2

Weights fitted from check-ins, with intervals and a "how much has this changed my model" view. Later: hierarchical priors so a new user starts from what others' data suggests, without sharing anyone's data.

AI NARRATOR V2

A weekly plain-language summary ("your Tuesdays are heavy because of three switches before 11:00"), generated from attribution data and rejected automatically if any number doesn't match it.

CALENDAR IMPORT LATER

Read an .ics file or a calendar export to build tomorrow's day in one step, then ask the user to rate complexity, since a calendar can't know that.

A SMALL VALIDATION STUDY LATER

With consent, a handful of volunteers rate real blocks for two weeks. Compare default and calibrated predictions against their TLX ratings, report error honestly, and publish the protocol and results.

PHYSIOLOGICAL SIGNALS RESEARCH

Heart-rate variability from a wearable as an optional extra input for calibration. Kept as research until it is clear it adds signal and not just noise and privacy risk.

RULED OUT ON PURPOSE

Team dashboards that rank people, automatic activity tracking, and employer-facing reports. They would turn a tool for understanding your own day into a tool for watching other people's.

APPENDIX A

GLOSSARY & DESIGN LANGUAGE

TERM	MEANING IN THIS PROJECT
Cognitive load	The demand a situation places on limited working memory. Here: the model's output L(t), an estimate, not a measurement.
Intrinsic load	Load from the task itself: how many elements must be held in mind at once `Sweller et al. 1998`.
Switch cost	The extra time and errors that come with changing tasks, even when the change is expected `Monsell 2003`.
Attention residue	Thoughts about a previous, unfinished task that carry into the next one `Leroy 2009`.
Working sphere	A unit of work with a shared goal, used in field studies of fragmented work `Mark et al. 2005`.
Open loop	A task started and not finished, which keeps drawing attention until it is done or planned.
NASA-TLX	A six-question workload scale: mental, physical and temporal demand, performance, effort, frustration. "Raw" TLX averages the six without weighting `Hart 2006`.
Lever	A what-if change to a day, written as a pure function from one day to another.
Stand-in	A parameter value we chose because the literature doesn't give a usable number at a one-minute scale. Flagged everywhere it appears.
Focus band	The middle zone, 0.3 to 0.65 by default. A display convention, moved by calibration.

COLOUR: ONE HUE PER DRIVER

COMPLEXITY	SWITCHING	ALERTS	OPEN LOOPS	DEADLINE	FATIGUE
#2F5BD3	#E5452F	#F2B233	#E0569B	#7B4FC9	#2FA37A

WHERE IT COMES FROM

Three references: the **Inside Out** infographic (one saturated colour per feeling, thought orbs, donut charts), the **"Standing on the shoulders of giants"** poster (layered flat profile heads, leader-line labels) and a **self-confidence slide deck** (warm beige paper, tall condensed capitals, black pill labels).

TYPE & SHAPES

Big Shoulders Display for headings, **DM Sans** for text, **DM Mono** for labels and numbers. Beige paper `#F6EAD8`, navy ink `#1C2240`. Rounded 4 mm corners, pill labels, and the profile head as the recurring mark. The app uses the same tokens, with a dark mode that keeps the six driver colours.

APPENDIX B

REFERENCES

Grouped by the driver or method they support. Week 1 of the build plan is reading each one in full and recording, in `params.ts`, exactly which finding each parameter leans on.

[1] Altmann, E. M., & Trafton, J. G. (2002). Memory for goals: An activation-based model. *Cognitive Science, 26*(1), 39–83.

[2] Ariga, A., & Lleras, A. (2011). Brief and rare mental "breaks" keep you focused: Deactivation and reactivation of task goals preempt vigilance decrements. *Cognition, 118*(3), 439–443.

[3] Boksem, M. A. S., & Tops, M. (2008). Mental fatigue: Costs and benefits. *Brain Research Reviews, 59*(1), 125–139.

[4] Cowan, N. (2001). The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences, 24*(1), 87–114.

[5] Fitz, N., Kushlev, K., Jagannathan, R., Lewis, T., Paliwal, D., & Ariely, D. (2019). Batching smartphone notifications can improve well-being. *Computers in Human Behavior, 101*, 84–94.

[6] Hart, S. G. (2006). NASA-Task Load Index (NASA-TLX); 20 years later. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting, 50*(9), 904–908.

[7] Hart, S. G., & Staveland, L. E. (1988). Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In P. A. Hancock & N. Meshkati (Eds.), *Human Mental Workload* (pp. 139–183). North-Holland.

[8] Leroy, S. (2009). Why is it so hard to do my work? The challenge of attention residue when switching between work tasks. *Organizational Behavior and Human Decision Processes, 109*(2), 168–181.

[9] Mark, G., González, V. M., & Harris, J. (2005). No task left behind? Examining the nature of fragmented work. *Proceedings of CHI 2005*, 321–330.

[10] Mark, G., Gudith, D., & Klocke, U. (2008). The cost of interrupted work: More speed and stress. *Proceedings of CHI 2008*, 107–110.

[11] Masicampo, E. J., & Baumeister, R. F. (2011). Consider it done! Plan making can eliminate the cognitive effects of unfulfilled goals. *Journal of Personality and Social Psychology, 101*(4), 667–683.

[12] Monsell, S. (2003). Task switching. *Trends in Cognitive Sciences, 7*(3), 134–140.

[13] Rogers, R. D., & Monsell, S. (1995). Costs of a predictable switch between simple cognitive tasks. *Journal of Experimental Psychology: General, 124*(2), 207–231.

[14] Rubinstein, J. S., Meyer, D. E., & Evans, J. E. (2001). Executive control of cognitive processes in task switching. *Journal of Experimental Psychology: Human Perception and Performance, 27*(4), 763–797.

[15] Stothart, C., Mitchum, A., & Yehnert, C. (2015). The attentional cost of receiving a cell phone notification. *Journal of Experimental Psychology: Human Perception and Performance, 41*(4), 893–897.

[16] Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science, 12*(2), 257–285.

[17] Sweller, J., van Merriënboer, J. J. G., & Paas, F. G. W. C. (1998). Cognitive architecture and instructional design. *Educational Psychology Review, 10*(3), 251–296.

[18] Teigen, K. H. (1994). Yerkes-Dodson: A law for all seasons. *Theory & Psychology, 4*(4), 525–547.

[19] Yerkes, R. M., & Dodson, J. D. (1908). The relation of strength of stimulus to rapidity of habit-formation. *Journal of Comparative Neurology and Psychology, 18*(5), 459–482.